Otimização

3.3 Métodos com derivadas

José Firmino Aguilar Madeira

Porquê usar derivadas

Os métodos de 3.2 só comparam valores de f e por isso precisam de muitas avaliações (Rosenbrock: n_f de 244 a 13 445). O gradiente indica diretamente a direção local de maior subida — e, portanto, $-\nabla f$ a de maior descida.

Quando usar

f diferenciável, derivadas disponíveis (analíticas, por diferenças finitas ou por diferenciação automática) e suficientemente fiáveis. Então podem ser muito mais rápidos.

Quando não usar

f descontínua ou não diferenciável, ou com ruído que torne as derivadas pouco fiáveis: voltar a 3.2.

Variáveis discretas: o gradiente usual não se aplica diretamente — é preciso outra formulação ou outro método.

Nesta secção: o que custa obter derivadas em $\mathbb{R}^n$, o que é uma direção de descida, e três métodos: gradiente (3.3.1), Newton (3.3.2), gradientes conjugados (3.3.3).

Derivadas numéricas em n dimensões

As fórmulas centrais de 2.6, coordenada a coordenada, com passos h_i:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x}+h_i\mathbf{e}_i) - f(\mathbf{x}-h_i\mathbf{e}_i)}{2h_i}, \qquad \frac{\partial^2 f}{\partial x_i^2} \approx \frac{f(\mathbf{x}+h_i\mathbf{e}_i) - 2f(\mathbf{x}) + f(\mathbf{x}-h_i\mathbf{e}_i)}{h_i^2},$$

$$\frac{\partial^2 f}{\partial x_i \partial x_j} \approx \frac{f(\mathbf{x}+h_i\mathbf{e}_i+h_j\mathbf{e}_j) - f(\mathbf{x}+h_i\mathbf{e}_i-h_j\mathbf{e}_j) - f(\mathbf{x}-h_i\mathbf{e}_i+h_j\mathbf{e}_j) + f(\mathbf{x}-h_i\mathbf{e}_i-h_j\mathbf{e}_j)}{4h_ih_j}$$

	avaliações de f
gradiente (n parciais)	$2n$
diagonal da Hessiana	$2n+1$ *
fora da diagonal $(n(n-1)/2)$	4 cada
Hessiana completa	$2n^2+1$

* reutiliza $f(\mathbf{x} \pm h_i\mathbf{e}_i)$ se já calculados para o gradiente.

- Para $n = 2$: gradiente 4, Hessiana 9. Para $n = 10$: 20 e 201. Para $n = 100$: 200 e 20 001.
- Mesmo compromisso de 2.6 na escolha de h_i (truncatura vs. arredondamento).
- Exemplo: Rosenbrock em $(-1{,}5; 2)$, $\nabla f = (-155; -50)$; com $h = 10^{-4}$ o erro é $6 \cdot 10^{-6}$.
- É por isto que os métodos de ordem 1 (3.3.1, 3.3.3) são atrativos quando n é grande.

Do 3.1: o que já sabemos sobre direções?

Revisão rápida (3.1):

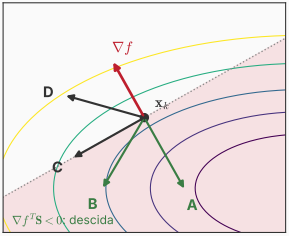
- $f(\mathbf{x}+\lambda\mathbf{S}) \approx f(\mathbf{x}) + \lambda\,\nabla f(\mathbf{x})^T\mathbf{S}$ para $\lambda > 0$ pequeno;
- $\mathbf{S}$ é direção de descida se $\nabla f(\mathbf{x})^T\mathbf{S} < 0$ (ângulo com ∇f maior do que 90°);
- para $\|\mathbf{S}\|$ fixa, $\nabla f^T\mathbf{S}$ é mínimo em $\mathbf{S} = -\nabla f$: descida mais rápida.

A novidade: uma família de direções de descida

Se M é definida positiva e $\nabla f(\mathbf{x}) \neq \mathbf{0}$,
$\mathbf{S} = -M\nabla f(\mathbf{x}) \;\Rightarrow\; \nabla f^T\mathbf{S} = -\nabla f^T M\,\nabla f < 0.$

$M = I$: gradiente (3.3.1). $M = H^{-1}$, $H = \nabla^2 f(\mathbf{x})$, *quando H é definida positiva*: Newton (3.3.2).

A mesma ideia unifica os dois métodos: mudar M é mudar a "métrica" em que se desce.



A figura do 3.1: A e B descem; C é tangente; D sobe.

O esquema comum aos três métodos

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{S}_k, \qquad \nabla f(\mathbf{x}_k)^T \mathbf{S}_k < 0$$

3.3 escolhe $\mathbf{S}_k$ | o cap. 2 escolhe λ_k

Nos métodos de descida — incluindo Newton quando H_k é definida positiva ou a direção é salvaguardada (3.3.2) — procura-se $\nabla f(\mathbf{x}_k)^T \mathbf{S}_k < 0$; o Newton puro não o garante se H_k for indefinida.

Método	Direção $\mathbf{S}_k$	Informação
3.3.1 Gradiente (Cauchy)	$-\nabla f(\mathbf{x}_k)$	ordem 1
3.3.2 Newton	$-[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$	ordem 2
3.3.3 Gradientes conjugados (Fletcher–Reeves)	$-\nabla f(\mathbf{x}_k) + \beta_k \mathbf{S}_{k-1}$	ordem 1 + memória

- O passo λ_k vem de uma pesquisa em linha sobre $\varphi(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{S}_k)$ (cap. 2). "Exata" = minimizar φ; "inexata" = descer o suficiente.
- Paragem: $\|\nabla f(\mathbf{x}_k)\| \leq \varepsilon_g$; $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| \,/\, \max(1, \|\mathbf{x}_k\|) \leq \varepsilon_x$; $k \geq k_{\max}$.
- Custo: n_g, n_H contados à parte de n_f (que inclui a pesquisa em linha); com derivadas numéricas tudo se converte em n_f: $2n$ por gradiente, $2n^2 + 1$ por Hessiana com gradiente.
- Sob hipóteses adequadas e pesquisa em linha apropriada, os três procuram *pontos estacionários*; a Hessiana aí, se não degenerada, classifica o ponto (semidefinida: inconclusivo).

Os contadores já feitos: `fminunc` e o SciPy

Em MATLAB, `fminunc` (quasi-Newton por defeito) devolve `out.funcCount` e `out.iterations`; com `'SpecifyObjectiveGradient'` o gradiente é analítico, senão é por diferenças finitas (gradientes progressivos do solver: $n+1$; nesta UC contamos centrais, $2n$ — 2.6).

```
opts = optimoptions('fminunc', 'SpecifyObjectiveGradient', true);
[x, fx, ~, out] = fminunc(@fg, x0, opts);  % fg devolve [f, g]
out.funcCount, out.iterations
```

A função completa (`fg`, que só calcula o gradiente quando `nargout > 1`) está no código complementar da UC.

O MESMO EM PYTHON

```
from scipy.optimize import minimize
r = minimize(f, x0, jac=grad, hess=hess,
             method='Newton-CG')
r.nfev, r.njev, r.nhev            # n_f, n_g, n_H
# com 'CG'/'BFGS': retirar hess= e r.nhev
```

Os contadores n_f, n_g, n_H vêm feitos. O caderno `cap3_comparacao.ipynb` reproduz a comparação dos sete métodos do fim do capítulo com as mesmas regras (mesmo problema, mesmo $\mathbf{x}_0$, mesmo critério, custo em avaliações).

Três maneiras de escolher $\mathbf{S}_k$

$-\nabla f$
usa a inclinação

$\rightarrow$

$-H^{-1}\nabla f$
inclinação + curvatura

$\rightarrow$

$-\nabla f + \beta_k \mathbf{S}_{k-1}$
inclinação + memória

	informação por iteração	custo com derivadas numéricas (+ as avaliações da pesquisa em linha)
Gradiente (3.3.1)	∇f	$2n$
Newton (3.3.2)	∇f e H	$2n^2 + 1$
Gradientes conjugados (3.3.3)	∇f e $\mathbf{S}_{k-1}$	$2n$

Comecemos pela informação mais barata: se só temos o gradiente, qual é a direção natural? — 3.3.1, o método do gradiente.

Referências e recursos

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2.ª ed., Springer, 2006 — caps. 2–3.
- Dennis, J. E., Jr., Schnabel, R. B., *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, SIAM (Classics in Applied Mathematics 16), 1996.

Recursos

- Código complementar: `fg` (f e gradiente). MATLAB: `fminunc`, `optimoptions('SpecifyObjectiveGradient')`, `out.funcCount`, `out.iterations`.
- Python: `scipy.optimize.minimize(..., method='Newton-CG')` (`r.nfev`, `r.njev`, `r.nhev`).
- Caderno `cap3_comparacao.ipynb`: comparação dos sete métodos do capítulo.